



UNIVERSIDAD
DE GRANADA



Cátedra de Inteligencia Artificial

Universidad de Granada – Unicaja

Agente Conversacional Bancario

Integrantes

Pablo Martín Palomino `pablomarpa@correo.ugr.es`

Pablo Reyes Sousa `nombre2@correo.ugr.es`

Natalia Yue Gallardo Pretel `nataliayue@correo.ugr.es`

Septiembre de 2026

Índice

1	Introducción y arquitectura general	2
1.1	Objetivos y enfoque de la solución	2
1.2	Arquitectura técnica	2
2	Interacción Natural y Experiencia de Usuario (UX)	2
2.1	Integración de voz y texto	2
2.2	Generación de respuestas en streaming	2
2.3	Interfaz adaptativa y accesible	2
3	Configuración, Generación y Seguridad de la Base de Datos	2
3.1	Una simulación realista y reproducible	2
3.2	Seguridad y Defensa en Profundidad (IA Responsable)	3
4	Motor de Inteligencia Artificial	3
4.1	Gestión del historial y prevención de desbordamiento de contexto	3
4.2	System Prompt dinámico y reglas de comportamiento	3
4.3	Uso de Tool Calling unificado	3
5	Operaciones Bancarias Seguras	3
5.1	Atajos de latencia	3
5.2	Flujo transaccional de Bizum	3
6	Consultas y Analítica Avanzada	4
6.1	Traducción del lenguaje natural a SQL para el histórico de movimientos . .	4
6.2	Enfoque híbrido: heurística y estadística	4
7	Motor Visual Generativo	4
8	Conclusiones y cumplimiento del baremo	4

1. Introducción y arquitectura general

1.1. Objetivos y enfoque de la solución

1.2. Arquitectura técnica

2. Interacción Natural y Experiencia de Usuario (UX)

2.1. Integración de voz y texto

2.2. Generación de respuestas en streaming

2.3. Interfaz adaptativa y accesible

3. Configuración, Generación y Seguridad de la Base de Datos

Para poner a prueba un asistente financiero inteligente, no basta con tener un buen modelo de lenguaje; necesitamos un escenario de datos rico, realista y, sobre todo, un entorno de ejecución completamente seguro. Por ello, el diseño de la base de datos se ha abordado desde dos frentes: la simulación de la vida financiera del usuario y la protección de su información.

3.1. Una simulación realista y reproducible

Aunque el enunciado del reto sugería el uso de un LLM para la generación masiva de datos, decidimos tomar un camino más controlado. Implementamos un generador determinista en Python (fijando una semilla de aleatoriedad) y limitamos el uso de la IA a la redacción del catálogo de comercios. Esta decisión nos dio una ventaja crucial: una base de datos 100 % reproducible. Esto es indispensable no solo para depurar el código sin que los datos cambien constantemente, sino para poder grabar una demostración en vídeo sólida y predecible.

Para enriquecer este entorno, aplicamos las siguientes estrategias en la generación de datos:

- **Histórico extendido de 24 meses:** Si generábamos solo un año de datos, un pago anual como el seguro del coche aparecería una única vez, haciendo matemáticamente imposible que el sistema detectara la recurrencia. Al ampliar la ventana a dos años, garantizamos que el asistente pueda responder a preguntas como “*¿cuándo pagué el seguro del coche?*”.
- **Patrones ocultos e inyección de ruido:** No queríamos ponérselo fácil a nuestro motor analítico. Intercalamos recibos mensuales exactos (alquiler), pagos bimestrales (agua) y gastos recurrentes de importe variable (luz). Además, programamos subidas de precio simuladas (como un incremento en la cuota de Netflix a mitad del histórico) para que el asistente realmente tuviera que esforzarse en detectar estos cambios entre todo el “ruido” de compras diarias en supermercados o gasolineras.

- **Convención de signos:** Diseñamos un esquema muy limpio con dos tablas (`cliente` y `movimientos`). Almacenar todos los gastos en negativo y los ingresos en positivo resultó ser un “truco” excelente para simplificar la vida del LLM, ya que le permite filtrar gastos en sus consultas SQL simplemente añadiendo la condición `importe < 0`.

3.2. Seguridad y Defensa en Profundidad (IA Responsable)

Dar a un modelo de lenguaje la capacidad de escribir código SQL sobre una base de datos real requiere medidas de seguridad estrictas. Para cumplir con los principios de una IA Responsable, hemos implementado un patrón de “defensa en profundidad”:

- **Aislamiento en modo lectura:** Cualquier consulta SQL que el asistente genere para responder dudas del usuario (el flujo *Text-to-SQL*) se ejecuta en una conexión a la base de datos configurada estrictamente como solo lectura (`mode=ro`). Si el LLM tiene una alucinación e intenta modificar algo, el propio motor de la base de datos lo bloquea físicamente.
- **Filtros y protección de contexto:** Antes de ejecutar el código del LLM, pasamos la consulta por una lista negra que bloquea palabras como `INSERT`, `UPDATE` o `DROP`. Además, limitamos drásticamente el número de filas que la base de datos puede devolver (50 filas máximo) para evitar desbordar la memoria del modelo y mantener una respuesta ágil.
- **Transaccionalidad para operaciones reales:** Cuando el usuario pide hacer una operación real, como enviar un Bizum, el sistema cambia las reglas. Estas acciones nunca utilizan SQL generado por la IA. En su lugar, el backend toma el control utilizando consultas parametrizadas seguras y bloques transaccionales estrictos (`BEGIN IMMEDIATE`) para evitar condiciones de carrera, asegurando que el dinero y el saldo se muevan siempre de forma exacta y coherente.

4. Motor de Inteligencia Artificial

4.1. Gestión del historial y prevención de desbordamiento de contexto

4.2. System Prompt dinámico y reglas de comportamiento

4.3. Uso de Tool Calling unificado

5. Operaciones Bancarias Seguras

5.1. Atajos de latencia

5.2. Flujo transaccional de Bizum

6. Consultas y Analítica Avanzada

6.1. Traducción del lenguaje natural a SQL para el histórico de movimientos

6.2. Enfoque híbrido: heurística y estadística

7. Motor Visual Generativo

8. Conclusiones y cumplimiento del baremo
